API's

API connects/ helps to communicate between two components using a defined set of rules, protocols, and data formats.

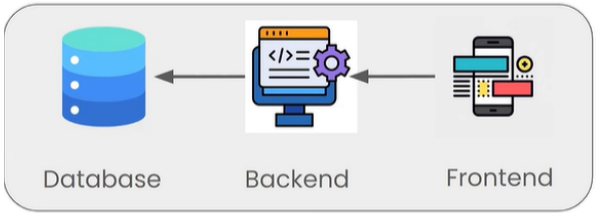
The protocol used here is HTTP
The response data format is JSON (Universal data format)

Analogy: Customer - Waiter - Chef

Need of APIs:

Pre-API era -

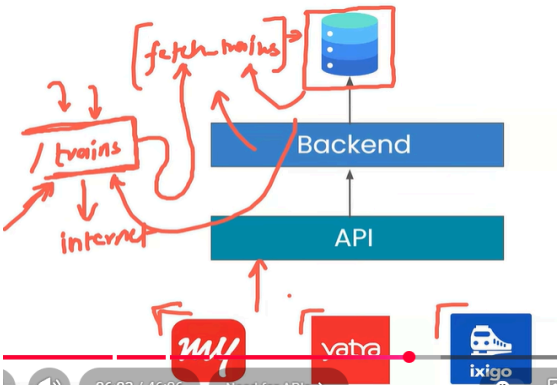
Without an API, the frontend and backend will be developed as a single application - hence tightly coupled and can communicate without an API —> Monolithic Architecture



Problem with MA: Is the tightly coupled nature

Solution:

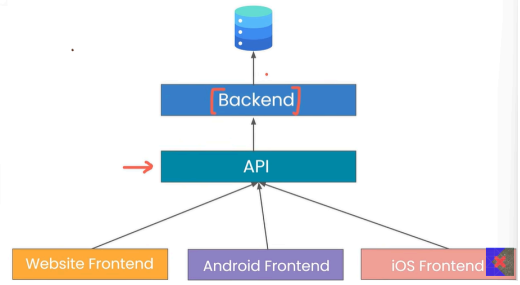
1. Decouple the application [Independent applications]
2. Add a layer of API [API is a set of endpoints - a public function]



Smartphones era -

For one MA application, we have to make 3 versions - Websites, Android, iPhone

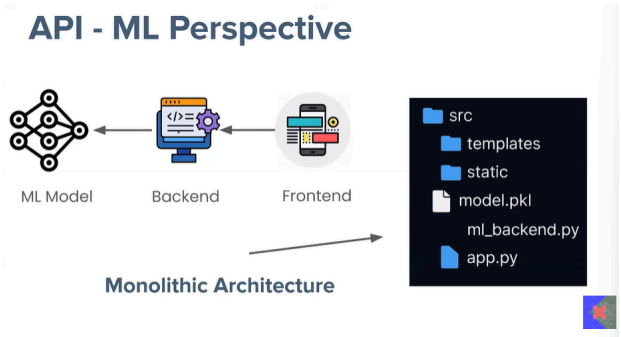
These 3 applications are independent



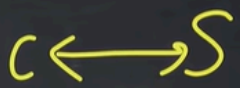
Solution: API Architecture

Single backend - connect with API - N no. of frontends

API - ML Perspective:



- REST is the most widely used API style for industry web applications.
- GraphQL is used when clients need flexible data fetching
- WebSocket is used for real-time communication
- gRPC is used between microservices
- SOAP is used in some enterprise systems
- Webhooks are used for event-based notifications.



REST API: Mobile Application to Backend

- REST normally represents data as resources and uses standard HTTP methods.
- GET Read data (GET /restaurants/101/menu)
- POST Create something (POST /orders)
- PUT Replace something
- PATCH Partially update something (PATCH /orders/501)
- DELETE Delete something (DELETE /cart/items/10)

GraphQL: Complex Customer Dashboard

Customer name, Last three orders, Reward points, Saved addresses, Recommended restaurants

- With REST, the application might call several endpoints:

GET /users/10

GET /users/10/orders

GET /users/10/rewards

GET /users/10/addresses

GET /users/10/recommendations

```
query { user(id: 10) {
name
rewardPoints
recentOrders(limit: 3)
savedAddresses { city }
recommendations { name rating }
}
```

WebSocket: Live Delivery Tracking

- REST can request the latest location:
- GET /orders/501/location

Where is the driver now?
Where is the driver now?
Where is the driver now?

```
GET /chat HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNZQ==

HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
```

gRPC: Communication Between Microservices

Order Service
Payment Service
Restaurant Service
Delivery Service
Notification Service



Webhook

- A webhook is an event-based HTTP callback.
- When an event occurs in Application A, it automatically sends an HTTP request to a predefined URL of Application B.

```
POST /webhook/payment
Content-Type: application/json
X-Webhook-Signature: a1b2c3d4
X-Webhook-Timestamp: 1720000000
{
  "event": "payment.success",
  "orderId": 1001,
  "amount": 50000
```

SOAP: Connecting to an Older Banking System

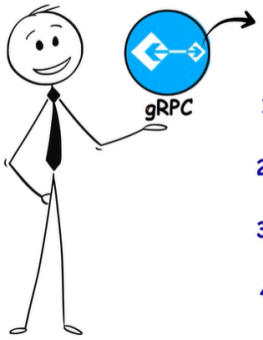
- A **strict, XML-based protocol** used for secure and reliable communication between applications

```
<soap:Envelope
  <soap:Body>
    <GetBalance>
      <AccountNumber>
    </GetBalance>
  </soap:Body>
</soap:Envelope>
```

REST - Independent, platform-independent
Sends text-based JSON over HTTP

SOAP - Old one, protocol-independent, used over http/s, SMTP, other protocols, built for error handling, security, transaction support, not lightweight/flexible but guarantees delivery, strict contracts, enterprise-grade reliability

Rpc - Remote procedure call
Instead of sending the raw data over the network, your app can directly call a function on another machine as if it were local
Earlier RPCs like XML-RPC and JSON-RPC had problems like it is slower, text-heavy, didn't scale well for today's massive data.
Hence, gRPC came - Google's high-performance modern take on RPC, built for speed, performance, and precision



four powerful communication patterns:

1. Simple request-response just like REST
2. Server streaming for live updates
3. Client streaming for sending continuous data
4. bidirectional streaming, where both sides can chat at once in real time

Uses Protocol Buffers or protobuf which compress data into a compact binary format that's lightning-fast to process

gRPC takes advantage of HTTP2 allowing multiple requests to run over a single connection at the same time.
7 to 10 times faster than REST; hence, it is used in netflix, uber, trading platforms

GraphQL - Graph Query Language created by Facebook
In the case of REST API, we either get too much data or too little data, which leads to over/underfetching
Whereas GraphQL gives one endpoint, one request.

The killer feature is real-time subscriptions + is self-documenting, with a built-in playground where we can test the queries instantly.

GitHub's entire API is built on GraphQL
Shopify and Pinterest use GraphQL in production

If you're building modern applications where performance matters, users are on mobile devices, and you want to give front-end developers the flexibility to request exactly what they need. GraphQL might just be ur new best friend.

Webhook -
With most APIs, your app is like someone constantly checking their mailbox. You walk outside, open it, see nothing, and walk back over and over again. That's how traditional APIs work. Your app has to ask the server every time it wants to know if something new happened. But webhooks flip that model completely. Instead of you asking, the API calls you. It's like the mailman ringing your doorbell the moment a letter arrives. Instant, direct, and efficient.

That's why webhooks are often called reverse APIs
GitHub fires webhooks when new code is pushed. Shopify triggers them when an order is placed. Slack and Discord bots rely on webhooks for commands and real-time reactions.

From automating workflows to keeping systems instantly in sync, web hooks are the invisible backbone of real time web development. Websockets API.

WebSocket - Gives a persistent 2-way communication line perfect for real-time applications

Unlike regular HTTP, where the client always initiates, WebSockets allow the server to push data to you the moment something happens. can send plain text, JSON for structured data, or even binary files like images and videos

WebARTc -
Web Real-Time Communication isn't just a single API. It's a full framework that enables direct peer-to-peercommunication between browsers or mobile apps.
The data doesn't need to flow through a central server. That's why WebRTC powers things like video calls, screen sharing, online gaming, and instant file transfers. All happening right inside your browser with no extra software.
Think about your last Zoom or GoogleMeet Call.
No server in the middle storing or processing your private conversation.
It's direct and real-time. Behind the scenes, WebRTC takes care of the messy networking details.
No server bottlenecks, faster communication, and smoother real-time experiences.